Campus Event Management & Feedback Analyzer

A PROJECT REPORT

Submitted by

Roll Number	Registration Number	Student Code	Student Name
26102410047	26102000169	BWU/MCA/26/052	Arnab Maity

in partial fulfillment for the award of the degree

Of

MASTER OF COMPUTER APPLICATIONS

IN

DEPARTMENT OF COMPUTATIONAL SCIENCES

BRAINWARE UNIVERSITY

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125



NOVEMBER 2026

Campus Event Management & Feedback Analyzer

Submitted by

Roll Number	Registration Number	Student Code	Student Name
26102410047	26102000169	BWU/MCA/26/052	Arnab Maity

in partial fulfillment for the award of the degree

of

MASTER OF COMPUTER APPLICATIONS

in

DEPARTMENT OF COMPUTATIONAL SCIENCES



BRAINWARE UNIVERSITY

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125

BRAINWARE UNIVERSITY

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125

[COMPUTATIONAL SCIENCES]

BONAFIDE CERTIFICATE

Certified that this project report **"Campus Event Management & Feedback Analyzer"** is the bonafide work of **"Arnab Maity"** who carried out the project work under my supervision.

SIGNATURE	**SIGNATURE**
DR. Jayanta Aich	MR. Rahul Kumar Ghosh
HEAD OF THE DEPARTMENT	**SUPERVISOR**
Computational Sciences	Assistant Professor
Brainware University	Computational Sciences

ACKNOWLEDGEMENT

With profound respect and deep gratitude, we express our sincere appreciation to our project supervisor, Mr. Rahul Kumar Ghosh, Assistant Professor, Computational Sciences, Brainware University, for his exceptional guidance, constant supervision, and invaluable suggestions throughout the course of this project. His expertise, constructive feedback, and patience have been instrumental in shaping the direction and successful completion of this work.

I extend my heartfelt thanks to Dr. Jayanta Aich, Head of the Department, Computational Sciences, Brainware University, for providing us with the necessary academic support, facilities, and encouraging environment that enabled us to carry out this project efficiently.

Roll No	Registration No	Student Code	Student Name	Signature
26102410047	26102000169	BWU/MCA/26/052	Arnab Maity	

ABSTRACT

This project presents a Python-based application that analyzes campus event management data and participant feedback from CSV files. The system reads event records and feedback entries, calculates key metrics such as average ratings, attendance percentages, registration counts, event popularity rankings, and feedback distribution, and generates visual charts for better insights. It is developed using modular Python scripts (main.py, analysis.py, charts.py, event.py) and relies on libraries such as pandas, matplotlib, and related data-processing tools.

A simple command-line interface allows users to load data, perform analysis, view statistics, and generate graphical reports without complex coding. The system identifies events with low ratings or poor attendance, computes overall campus event statistics, and produces clear visualizations. This reduces manual calculation effort for event organizers and administrators and demonstrates how Python data analysis and visualization techniques can be combined into a practical campus management tool.

TABLE OF CONTENTS

Chapter	Title	Page No.
1	Introduction	1
2	Objective	2
3	Planning	3
4	Requirement Analysis	5
5	System Flow	6
6	Proposed Design	13
7	Experimental Result	15
8	Future Scope	18
9	Conclusion	19

LIST OF FIGURES

Figure No.	Title	Page No.
5.1	Context Level Diagram	6
5.2	Data Flow Diagram Level 0	7
5.3	Data Flow Diagram Level 1	8
5.4	Use Case Diagram	9
5.5	Activity Diagram	10
5.6	Sequence Diagram	11
5.7	Flow Chart	12
7.1	Data Loading	15
7.2	Feedback Analysis	15
7.3	Analysis Stage	16
7.4	Bar Chart Generation	16
7.5	Histogram Generation	17
7.6	Event Participation Generation	17

LIST OF SYMBOLS, ABBREVIATIONS, AND NOMENCLATURE

Symbol / Abbreviation	Description
CSV	Comma Separated Values (file format used for event & feedback data input)
DFD	Data Flow Diagram
CLD	Context Level Diagram
UML	Unified Modeling Language
GUI	Graphical User Interface
PDF	Portable Document Format
%	Percentage (used for Attendance / Rating Percentage)
Avg	Average (used for Average Rating / Feedback Score)
ID	Identifier (Event ID / Feedback ID)
	Data flow / Message direction (in DFD and Sequence Diagram)
/ ▭	Process / Activity symbol (in Flow Chart and Activity Diagram)
◇	Decision symbol (in Flow Chart and Activity Diagram)
▭ (open)	Data Store (in DFD)
Actor	External entity / User (in Use Case and Context diagrams)

CHAPTER 1: INTRODUCTION

Traditional campus event management and feedback collection relies on manual registers, spreadsheets, and time-consuming calculations. Event organizers often struggle to quickly identify events with low attendance or poor participant feedback, and generating summary reports or visual insights requires additional effort. With the availability of Python data analysis libraries, it is now possible to automate large parts of this process.

This project focuses on building a system that reads event and feedback data from CSV files, processes the records, calculates important metrics, and produces both textual statistics and graphical charts. The idea is simple: the user provides structured data files, and the program computes average ratings, attendance percentages, registration counts, event rankings, feedback distribution, and campus-level summaries, then visualizes the results.

The system uses a modular design with separate files for data modeling, analysis logic, chart generation, and the main program entry point. Behind the interface, the workflow follows clear steps: load data, validate records, compute performance and feedback metrics, generate visualizations, and present a consolidated report. The project demonstrates how data processing, statistical calculation, and visualization can be combined into a lightweight, easy-to-understand campus analytics tool suitable for students and beginners.

CHAPTER 2: OBJECTIVE

The objective of this project is to develop a Python-based Campus Event Management & Feedback Analyzer that enables users to process event and feedback records from CSV files and obtain meaningful operational and satisfaction insights. The system aims to provide a modular, automated workflow by integrating data loading, metric calculation, and visualization components. Users can load event and feedback data, compute individual and campus-level statistics (average ratings, attendance percentage, registration counts, event rankings), identify under-performing or low-rated events, and generate charts without requiring advanced programming knowledge. By combining pandas for data handling, custom analysis functions, and matplotlib-based visualizations, the project intends to simplify event monitoring, reduce manual calculation effort, and offer an accessible end-to-end event performance and feedback analysis pipeline.

CHAPTER 3: PLANNING

The planning phase outlines the step-by-step approach used to design, organize, and implement the Campus Event Management & Feedback Analyzer. Proper planning ensures that the workflow remains efficient, modular, and easy for beginners to understand and use.

1. Requirement Identification

- Understand the need for a simple Python tool to analyze campus events and feedback from CSV data.
- Identify the target users (event organizers, administrators, students, beginners).
- Define the core requirement: CSV data analysis of events & feedback statistics + charts.

2. System Design Planning

- Decide to use pandas for data handling and matplotlib for visualization.
- Choose a modular structure with separate files for Event class, analysis logic, chart generation, and main program.
- Plan the workflow from data loading to final report and charts.
- Select supporting libraries listed in requirements.txt.

3. Pipeline Structure

Design a sequential pipeline with the following steps:

- Data loading from events.csv and feedback.csv
- Event / Feedback object creation / data validation
- Rating and attendance metric calculation
- Event ranking and status determination
- Statistical summaries
- Chart generation
- Result display

4. File & Module Planning

- main.py – Entry point and user interaction / menu

5. Code Structure Planning

- Break the code into clear, reusable sections:
  - Data loading and validation
  - Metric calculation functions
  - Filtering of low-rated / low-attendance events
  - Chart plotting functions
  - Main menu / command-line interface

6. Testing & Validation Plan

- Test with sample CSV data.
- Verify correct average rating, percentage, and ranking calculations.
- Check chart generation for different metrics.
- Validate handling of edge cases (missing data, zero attendance, etc.).

7. Scalability & Improvement Planning

Plan for expandable features such as:

- Support for multiple campuses or departments
- Export of reports to PDF or Excel
- Interactive GUI (Tkinter or Streamlit)
- Predictive analysis using simple machine learning
- Database integration instead of CSV

CHAPTER 4: REQUIREMENT ANALYSIS

Functional Requirements

1. Data Input

- The system must read event and feedback records from CSV files (events.csv, feedback.csv).
- Each record typically includes event identifiers, registration counts, attendance, ratings, and comments.
- Input is handled through Python file reading and pandas DataFrame operations.

2. User Interaction

- A simple command-line interface or menu-driven program in main.py.
- Options to load data, run analysis, view statistics, generate charts, and exit.
- Users should not need to write code to obtain results.

3. Model and Framework Requirements

The project depends on the following Python libraries (as listed in requirements.txt):

- pandas (data loading and manipulation)
- matplotlib (chart generation)
- Other standard libraries (os, csv, etc.) as needed

4. Analysis Requirements

For every event / feedback record:

- Calculate average rating / satisfaction score.
- Calculate attendance percentage.
- Rank events by popularity or rating.
- Determine status (successful / needs attention) based on rating and attendance thresholds.
- Identify events with low ratings or poor attendance.

5. Visualization Requirement

- Generate bar charts, pie charts, histograms or other suitable plots for rating distribution, attendance ranges, event comparison, etc.
- Charts should be saved or displayed clearly.

6. Output Organization

The system must:

- Display textual summaries and statistics on the console.
- Generate and save chart images.
- Provide a clear final report of campus-level and individual event insights.

CHAPTER 5: SYSTEM FLOW

Context Level Diagram (CLD):

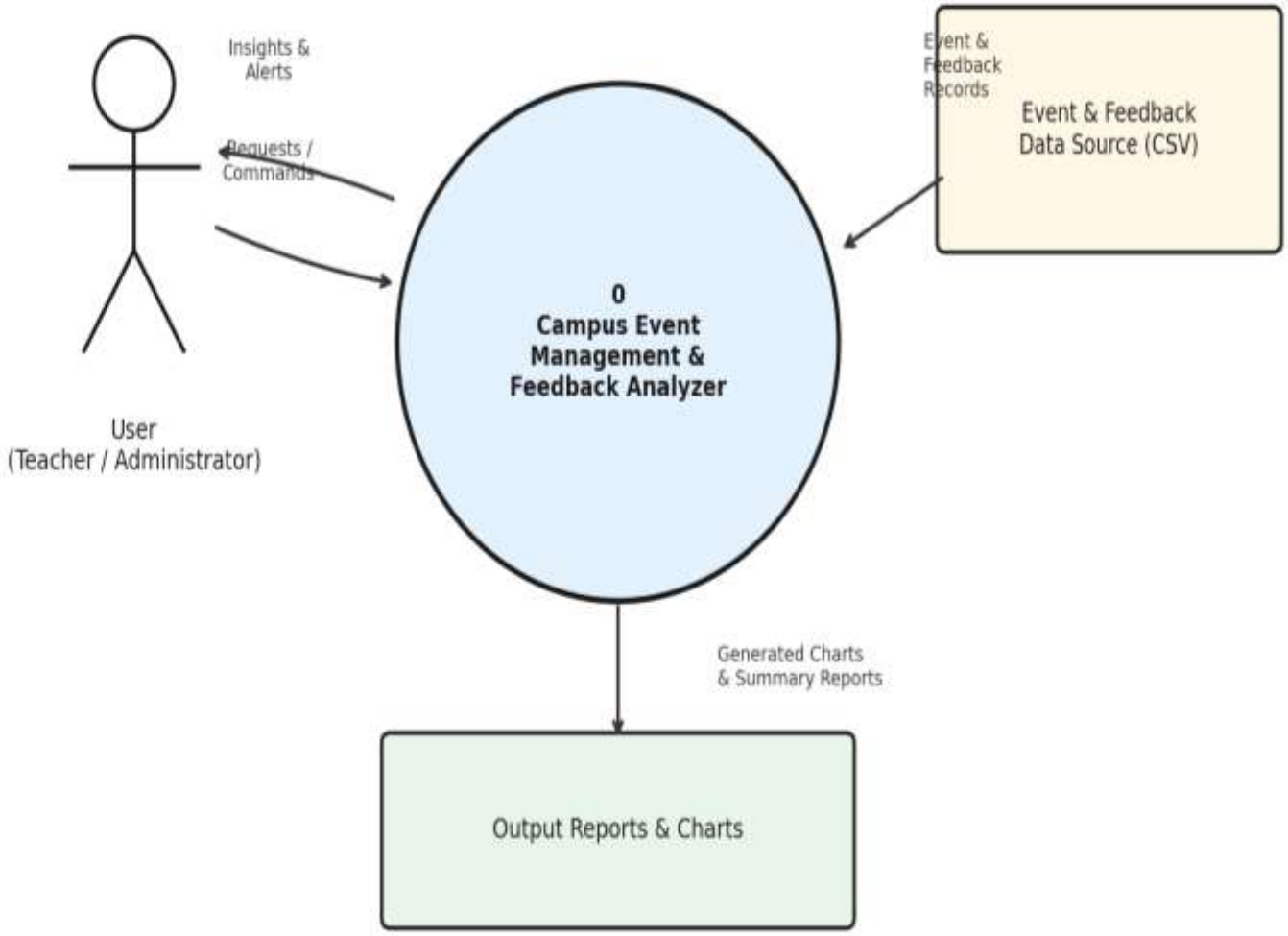


Fig 5.1: Context Level Diagram

It shows the overall system as a single process interacting with external entities (User and CSV data sources) and producing reports/charts.

Data Flow Diagrams (DFD):

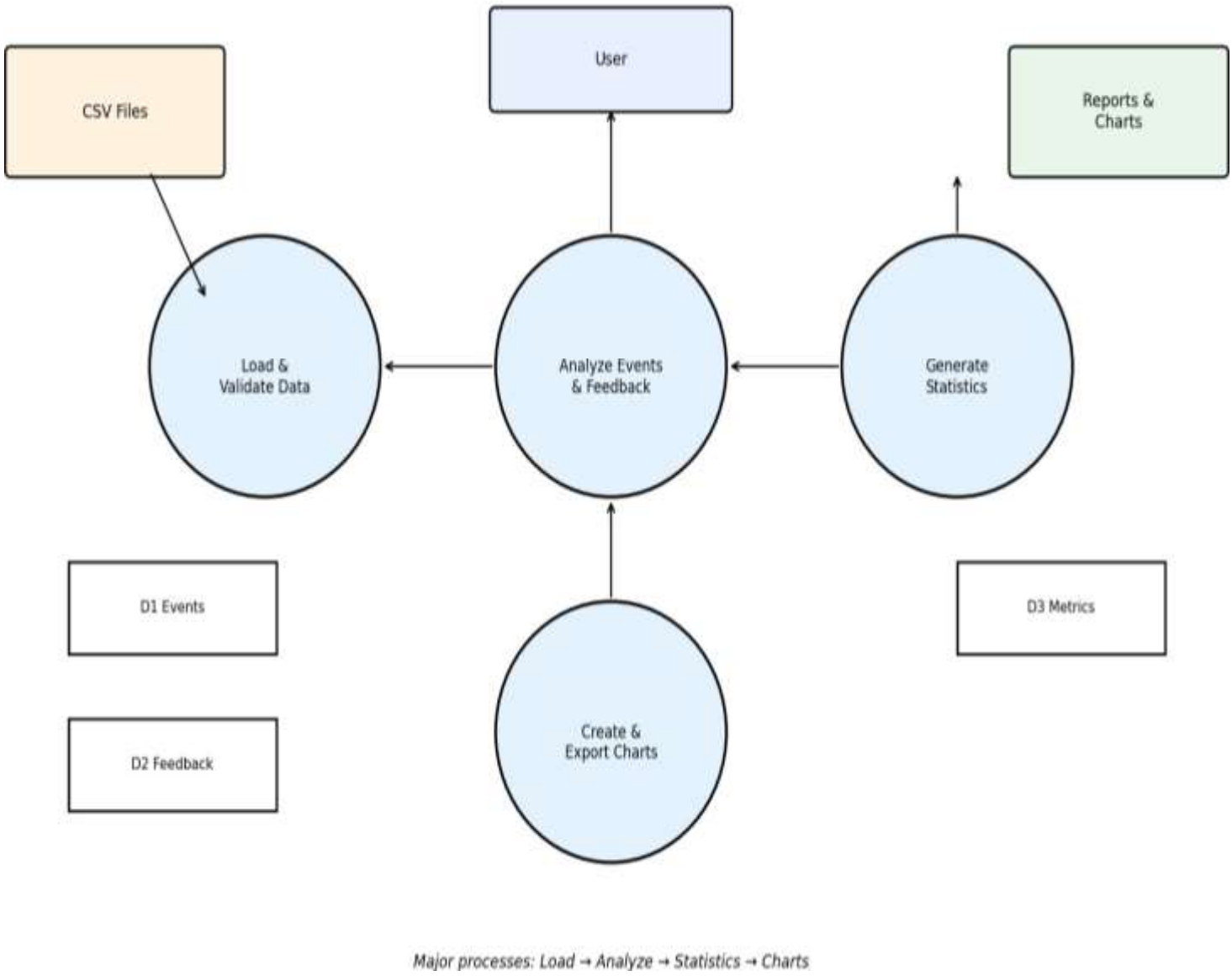


Fig 5.2: Data Flow Diagram Level 0

It illustrates the major processes (Load Data, Analyze, Generate Statistics, Create Charts) along with data stores and high-level data flow.

Data Flow Diagrams (DFD):

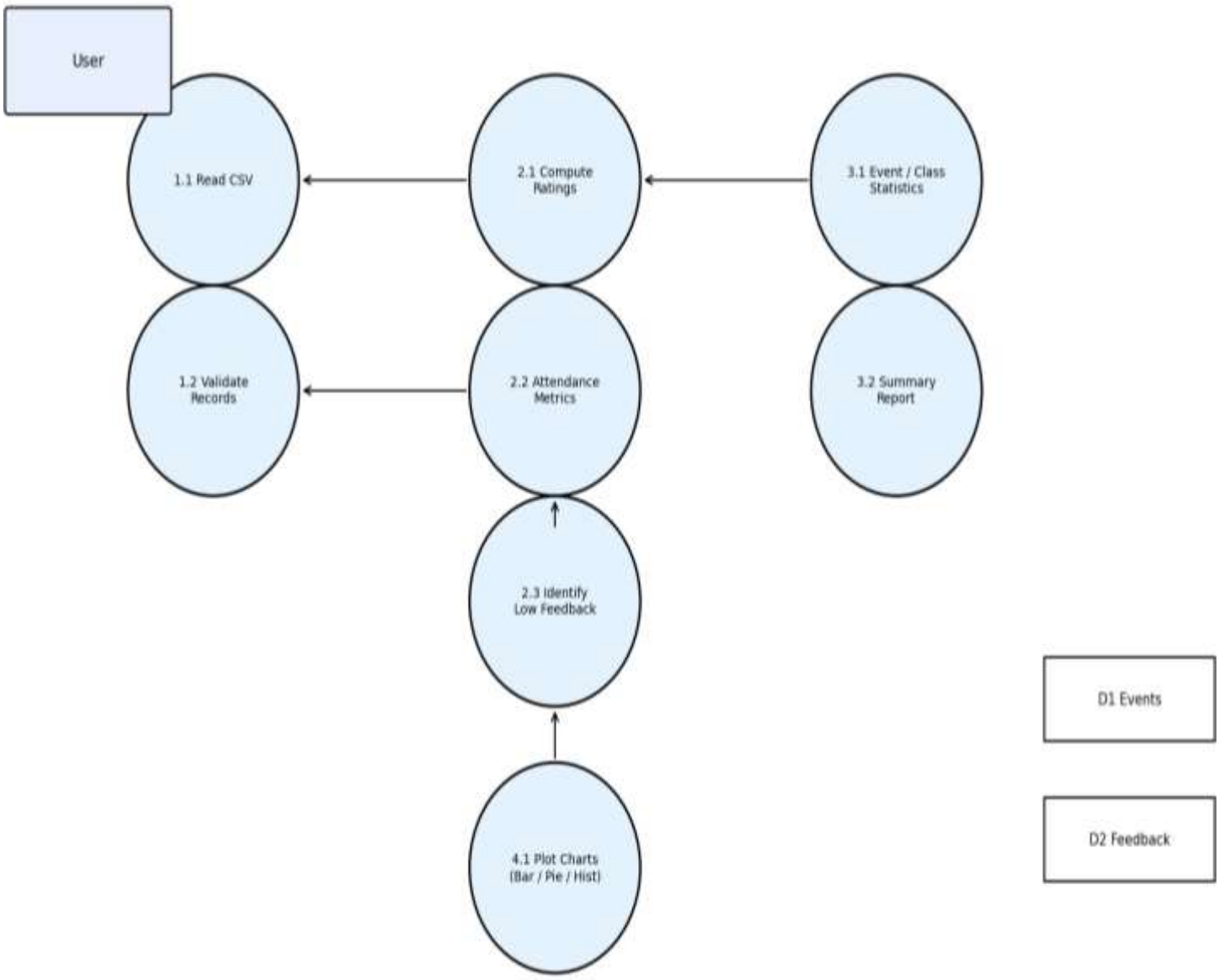


Fig 5.3: Data Flow Diagram Level 1

It breaks down the Level 0 processes into detailed sub-processes for data loading, metric calculation, statistics generation, and chart creation.

Use Case Diagram:

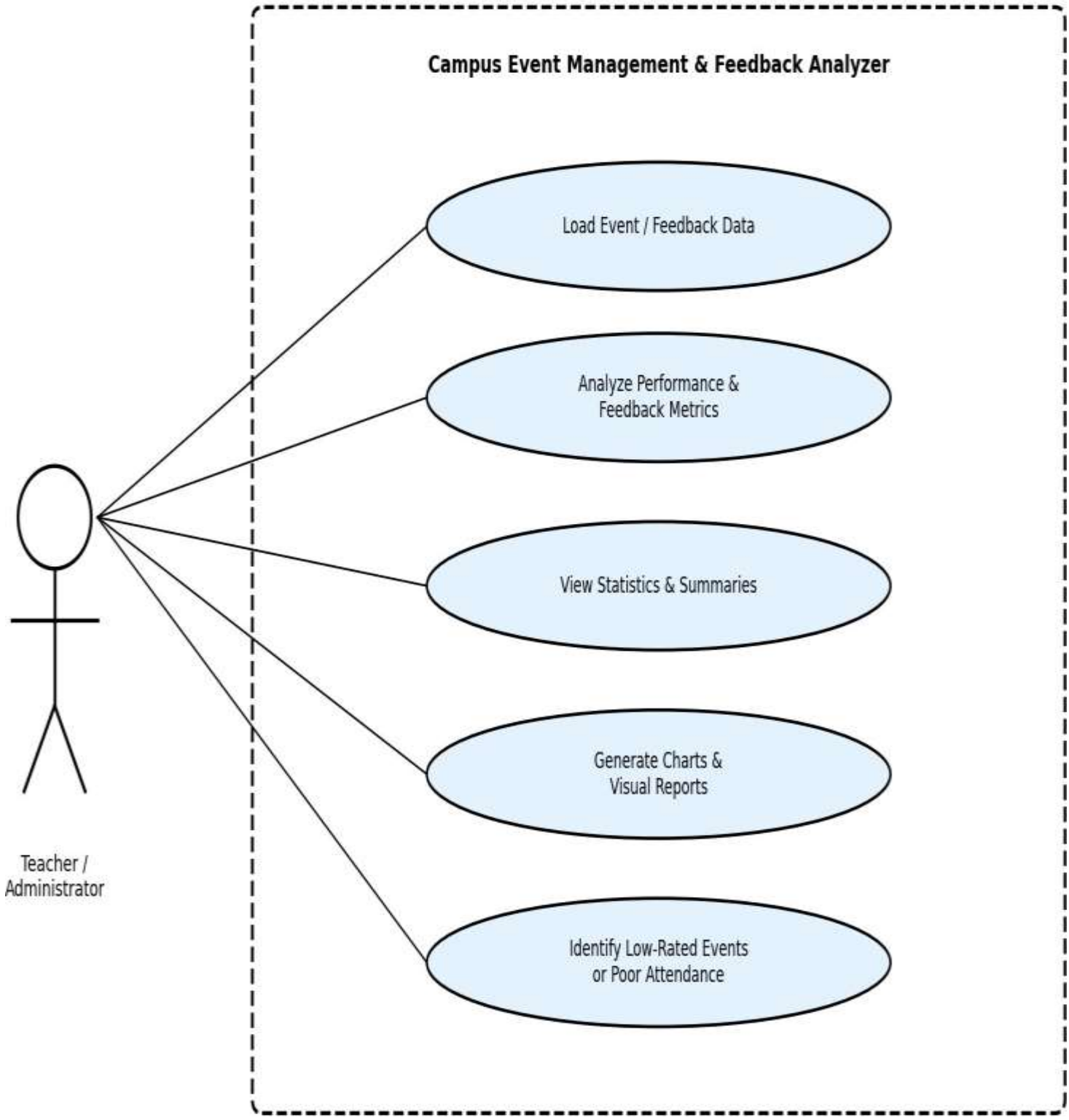


Fig 5.4: Use Case Diagram

It depicts the interactions between the Teacher/Administrator and the system's key functionalities such as loading data, analyzing performance, and generating reports.

Activity Diagram:

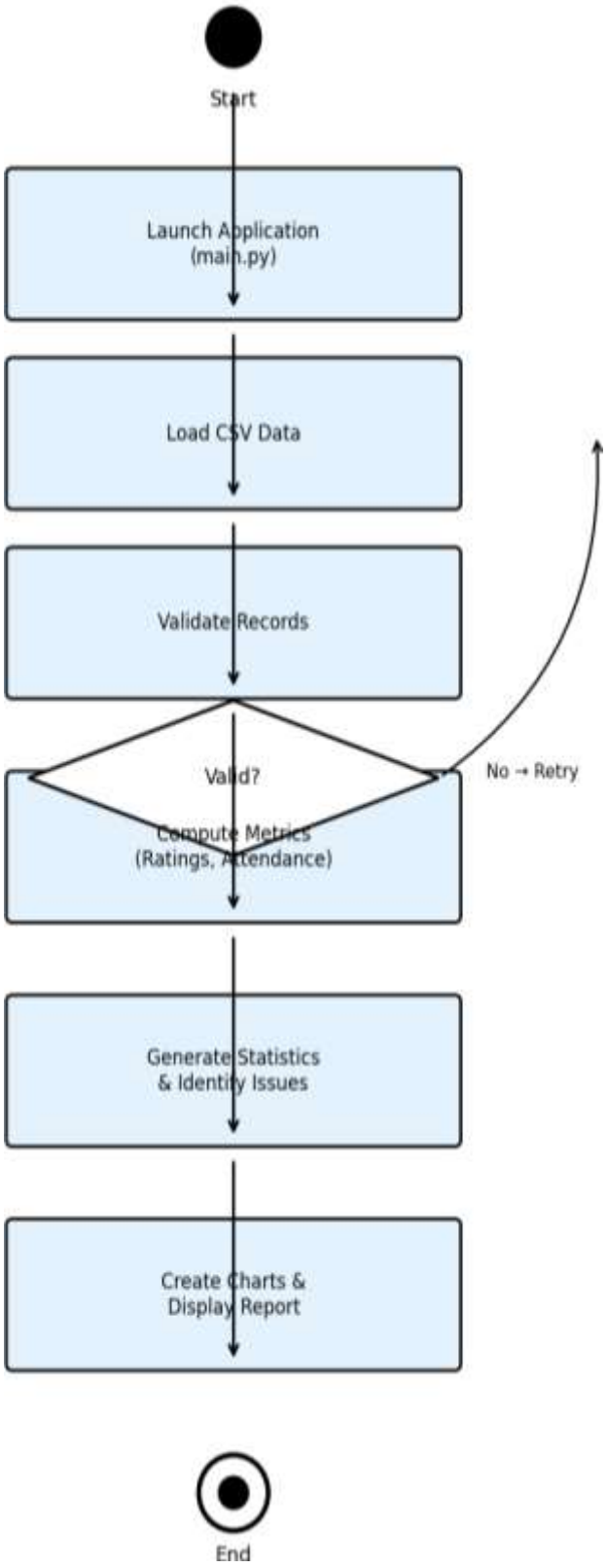


Fig 5.5: Activity Diagram

It represents the step-by-step workflow of the system from starting the application to displaying the final results, including decision points for data validation.

Sequence Diagram:

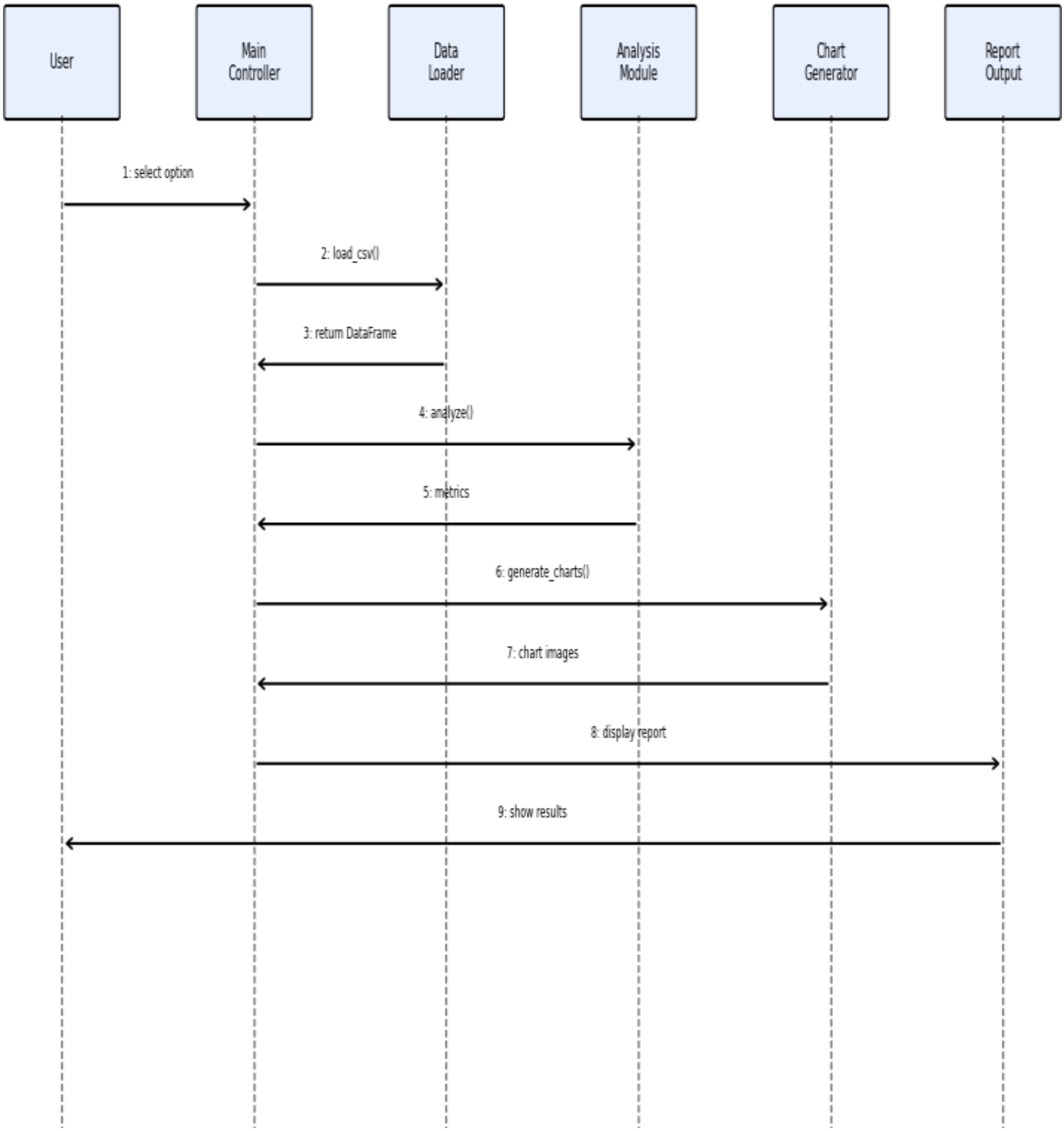


Fig 5.6: Sequence Diagram

It shows the chronological sequence of messages exchanged between the User, Main Controller, Data Loader, Analysis Module, Chart Generator, and Report Output.

Flow Chart:

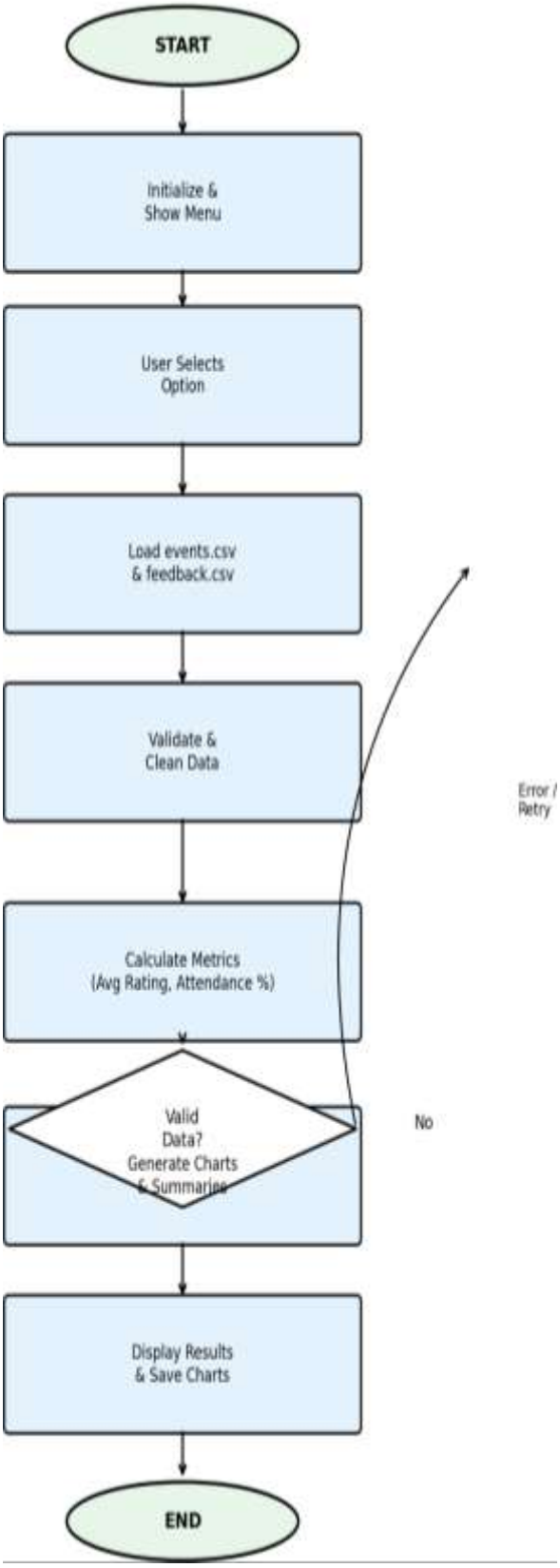


Fig 5.7: Flow Chart

It presents the complete sequential logic of the system using standard flowchart symbols, including decision points for data validation and the path to final report generation.

CHAPTER 6: PROPOSED DESIGN

This chapter explains how the system processes event and feedback data and produces analysis results and charts. The method follows a sequential pipeline that begins with CSV input and ends with statistics and visualizations. The overall workflow is shown in the diagrams, and each stage is described in detail.

The application loads event records from events.csv and feedback records from feedback.csv, creates structured data objects, computes rating and attendance metrics, filters important cases, and generates charts. This method allows users to obtain insights from campus event data without manual calculations or complex tools.

The process includes the following key stages:

1. Data loading from CSV
2. Event / Feedback data modeling
3. Performance and feedback analysis
4. Statistical summary generation
5. Chart creation and display
6. Final report presentation

Step 1: Data Loading

The program reads the events.csv and feedback.csv files using pandas and prepares clean datasets for analysis.

Step 2: Event Modeling

An Event class represents individual records with attributes such as name/ID, registrations, attendance, and ratings. Objects or DataFrame rows are created for further processing.

Step 3: Analysis

The analysis module performs calculation of average ratings, attendance percentage, event ranking, status determination (based on rating and attendance thresholds), identification of events needing attention, and campus-level aggregates.

Step 4: Chart Generation

The charts module uses matplotlib to produce visual representations such as rating distribution pie/bar charts, attendance histograms, performance comparison plots, and scatter views (attendance vs rating) when applicable.

Step 5: Main Program Flow

main.py orchestrates the process: loads data, calls analysis functions, triggers chart generation, and presents results through a simple menu or sequential execution.

Step 6: Final Output

Users receive console statistics, lists of low-rated or low-attendance events, and generated chart images. This supports both verification and decision-making.

REPRESENTATIVE CODE STRUCTURE (based on project files)

- **event**: Defines the Event class or data structure.
- **analysis**: Contains functions for calculating averages, percentages, rankings, status, and summaries.
- **charts**: Contains functions that generate and save/display matplotlib charts.
- **main.py**: Entry point that loads data, invokes analysis and chart modules, and handles user interaction.
- **events.csv / feedback.csv**: Sample datasets used for testing and demonstration.
- **requirements.txt**: Lists required packages (pandas, matplotlib, etc.).

This methodology is effective because data processing is automated and repeatable, modular design makes the code easy to understand and extend, visualizations improve insight communication, and no external specialized software is required beyond standard Python libraries. The workflow is suitable for undergraduate-level implementation while demonstrating practical data analysis techniques.

CHAPTER 7: EXPERIMENTAL RESULT

The results focus on how the CSV data was processed, how performance and feedback metrics were calculated, how statistics were produced, and how charts were generated. Screenshots of console outputs, charts, and sample reports can be inserted in the respective places.

1. Data Loading Stage

The system successfully reads the events.csv and feedback.csv files and prepares the data for analysis.

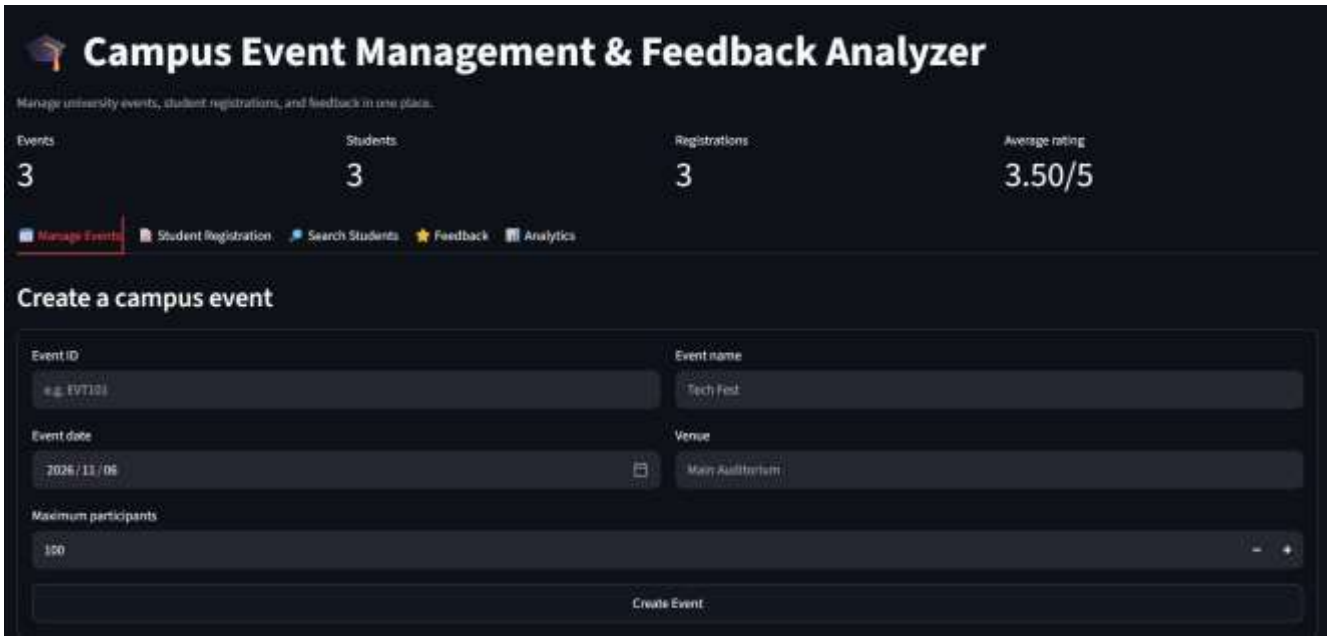


Fig 7.1: Data Loading Stage

2. Performance & Feedback Analysis

Average ratings, attendance percentages, and status are calculated for each event and for the campus overall.

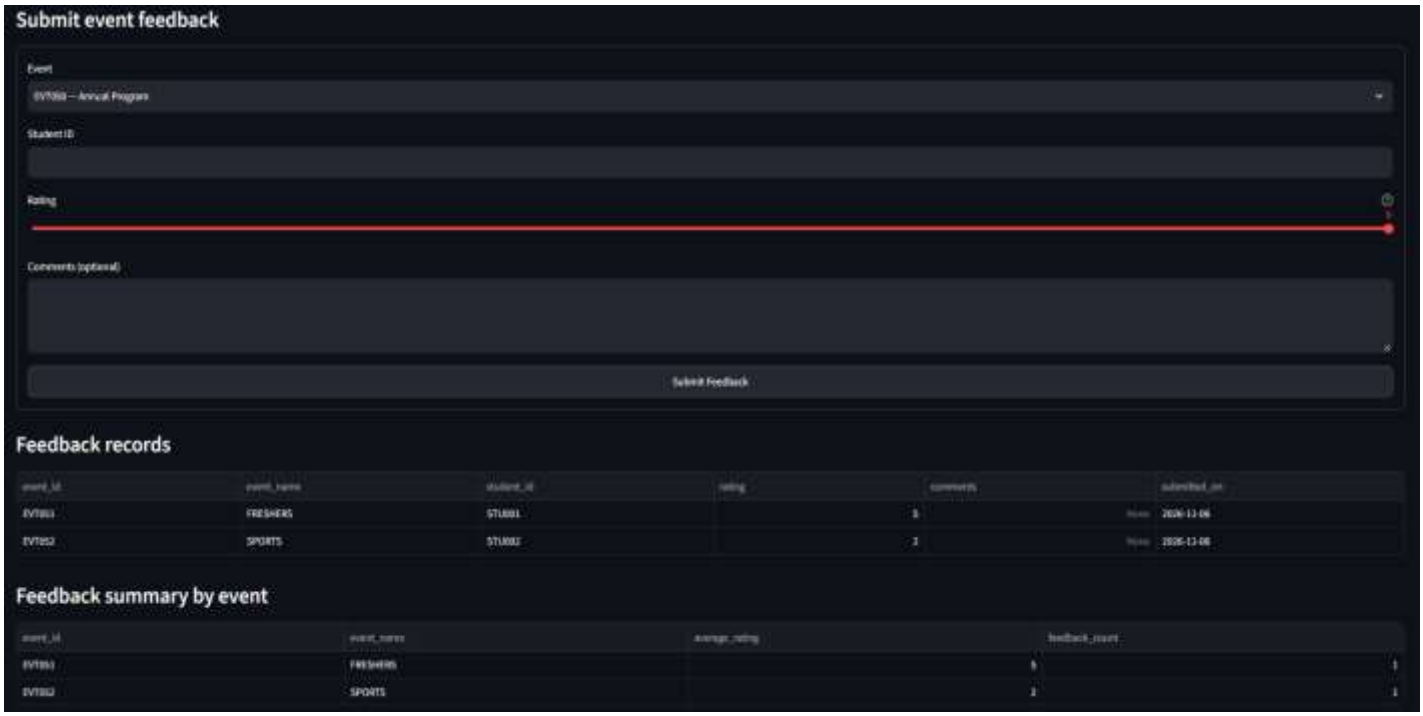


Fig 7.2: Feedback Analysis

3. Attendance & Rating Analysis

Attendance percentages and average ratings are computed; events falling below thresholds are identified.

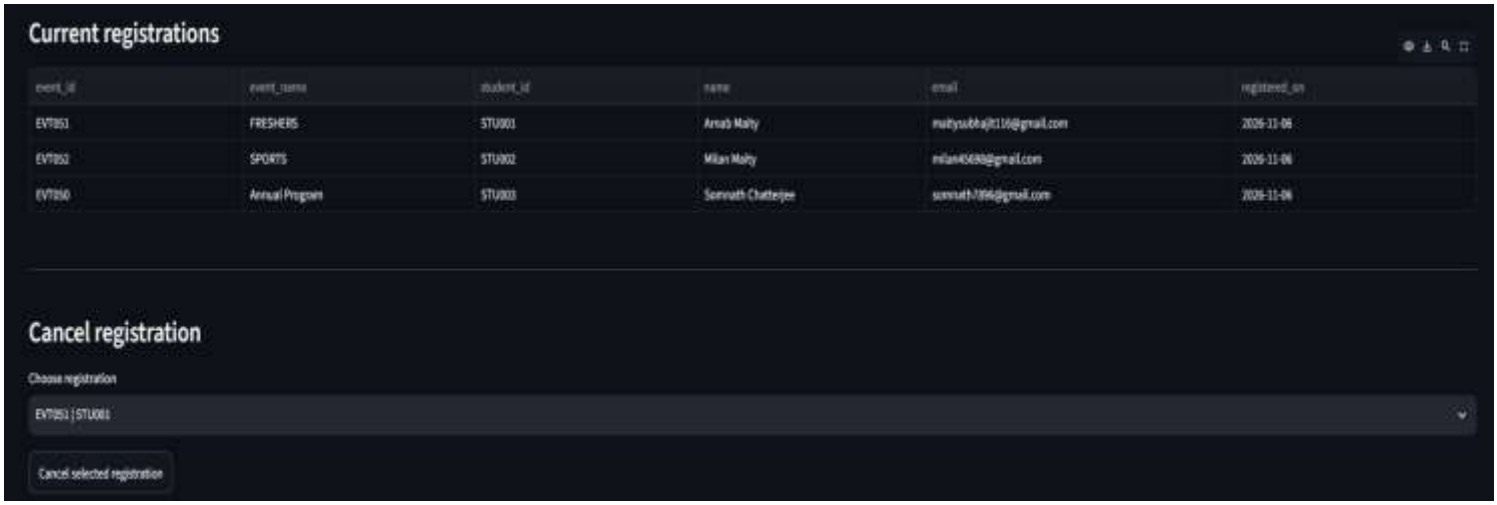


Fig 7.3: Analysis Stage

4. Charts Generation

Visual charts (rating distribution, attendance ranges, event comparison) are produced and saved.

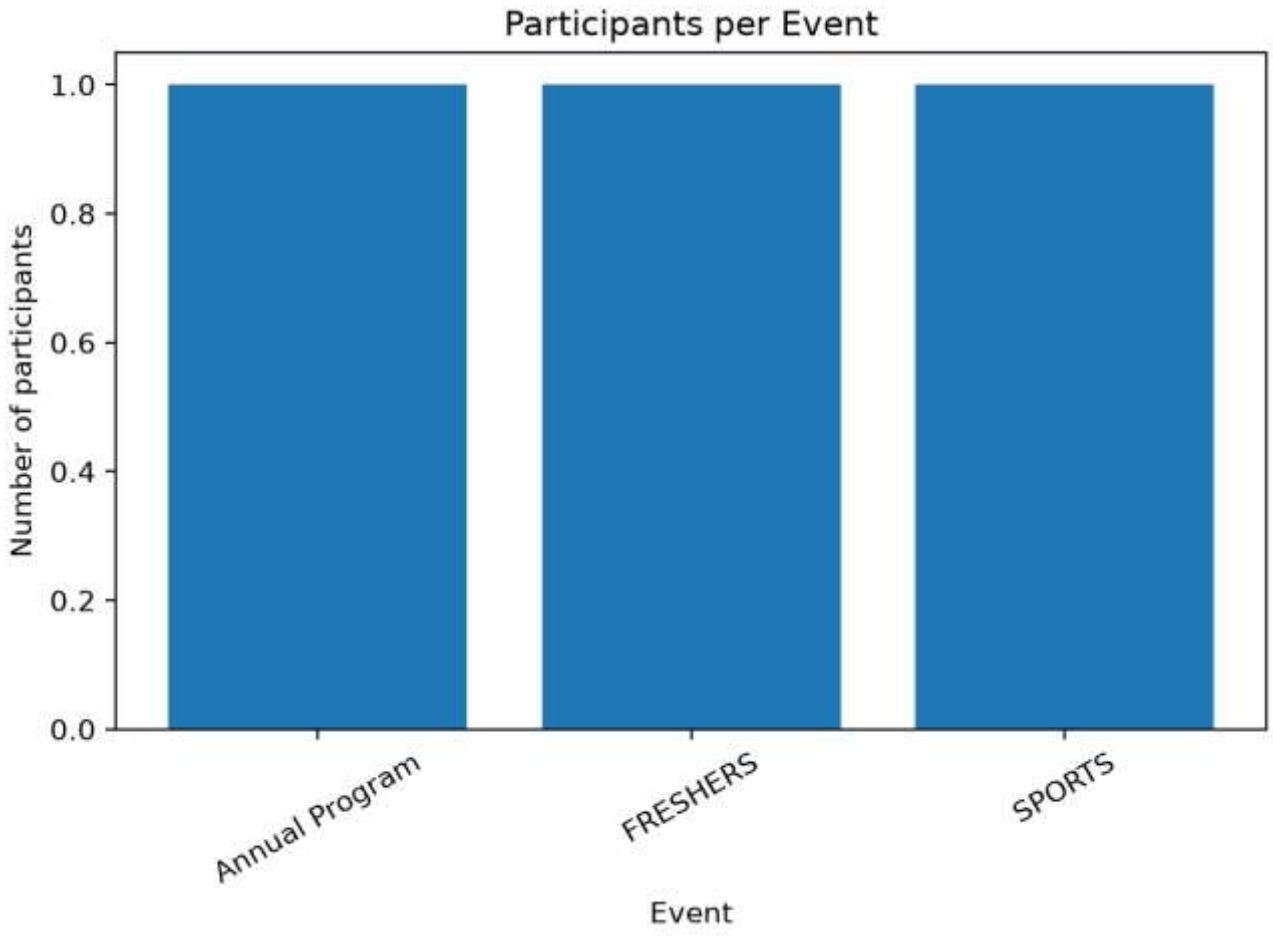


Fig 7.4: Bar Chart Generation

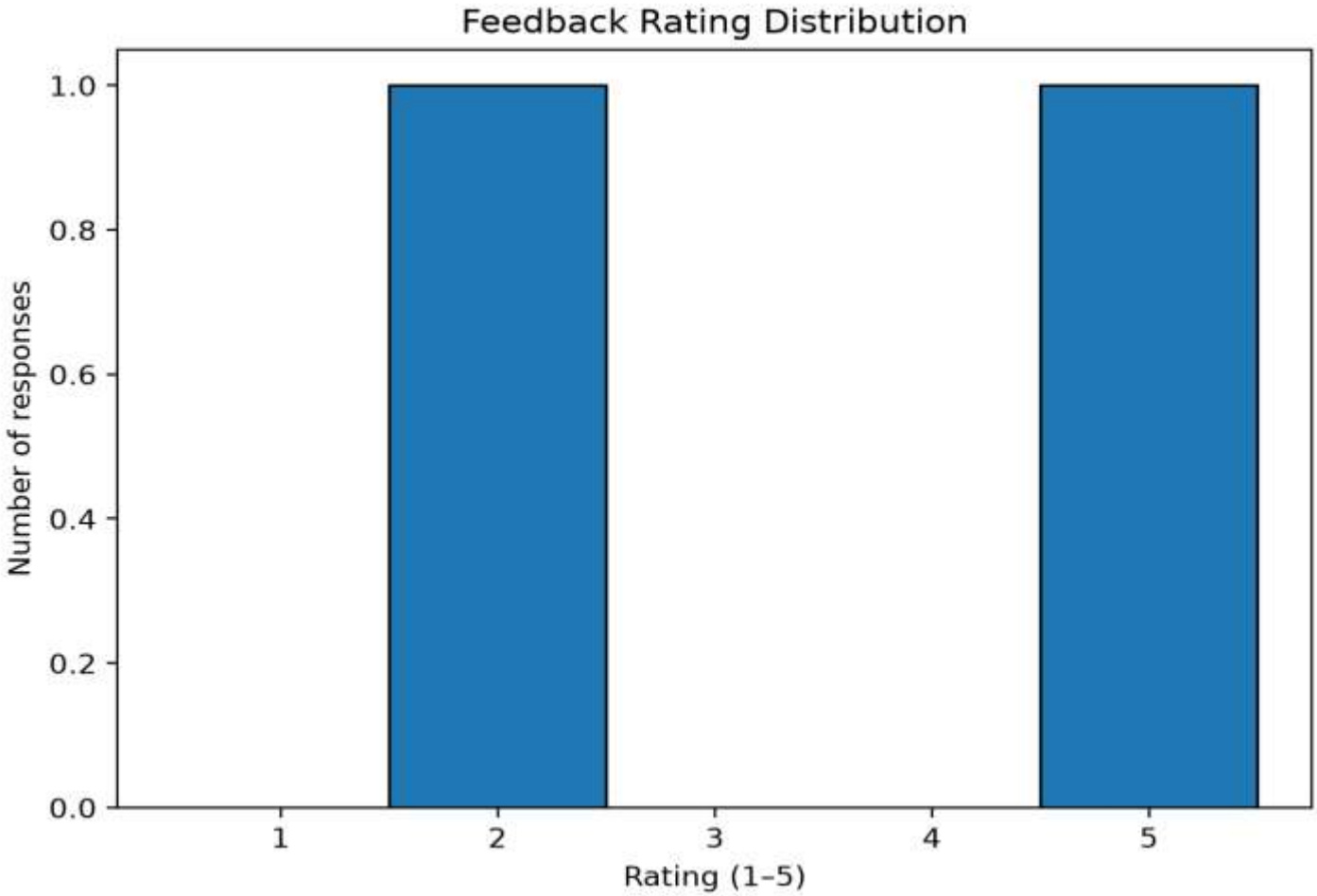


Fig 7.5: Histogram Generation

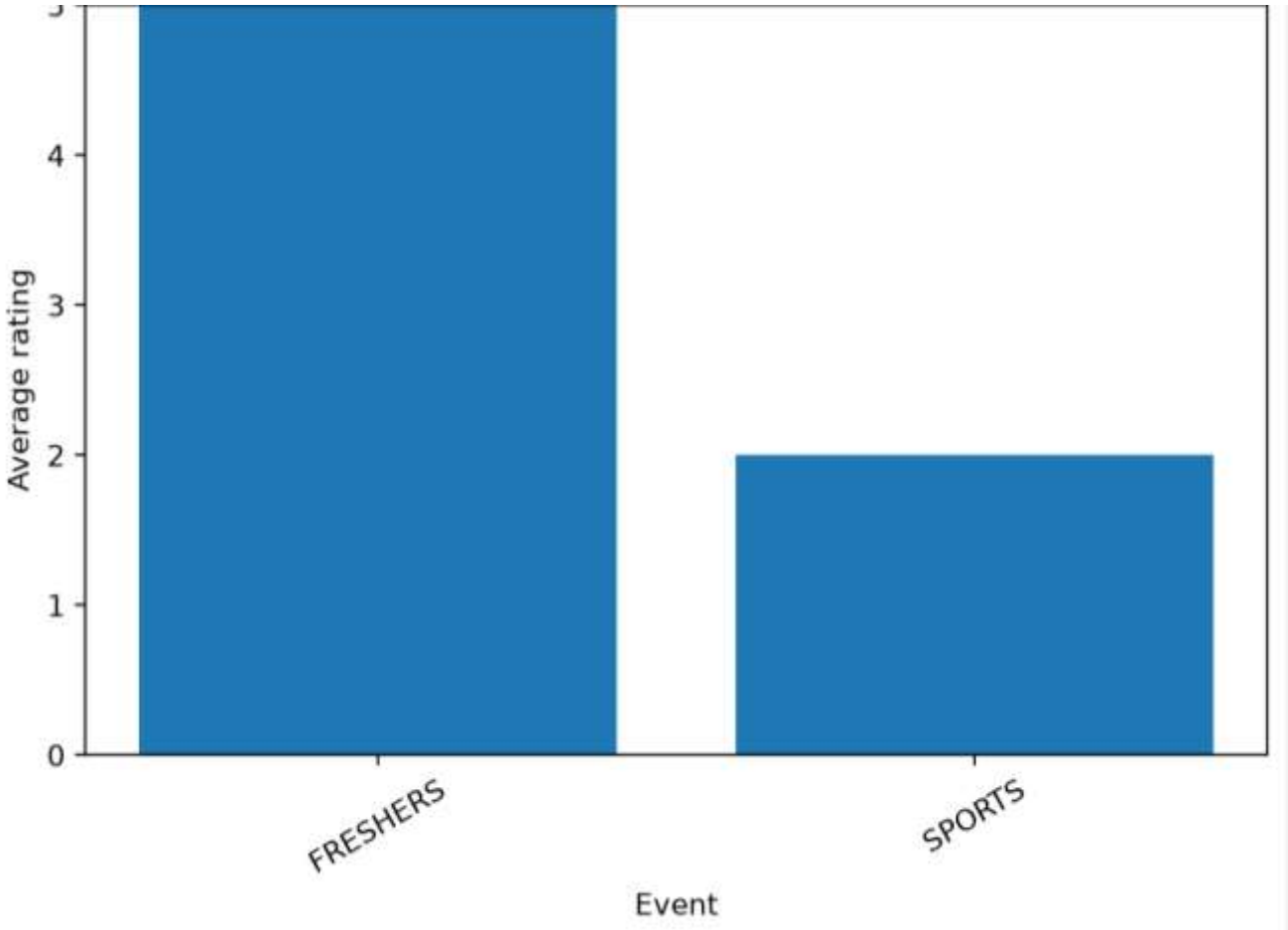


Fig 7.6: Event Participation Graph

CHAPTER 8: FUTURE SCOPE

Several improvements can be added in later versions of the project:

1. Enhanced User Interface

Develop a graphical user interface (Tkinter, Streamlit, or web-based dashboard) for easier data upload, interactive filtering, and real-time visualization.

2. Multi-Campus & Multi-Department Support

Extend the data model to handle multiple campuses, departments, continuous assessment components, and semester-wise tracking.

3. Report Export Features

Add options to export analysis results and charts to PDF, Excel, or PowerPoint for easy sharing with administrators and stakeholders.

4. Predictive Analytics

Incorporate simple machine learning models to predict future event success or identify at-risk events based on historical patterns.

5. Database Integration

Replace or supplement the CSV input with a proper database (SQLite/MySQL) for larger datasets and multi-user access.

6. Notification & Alert System

Add automatic alerts or email notifications for events with critically low attendance or declining feedback scores.

7. Advanced Visualizations

Include more sophisticated charts, heatmaps, trend lines over time, and interactive dashboards.

CHAPTER 9: CONCLUSION

The project successfully showcases a complete, end-to-end workflow for analyzing campus event performance and participant feedback from structured data through an automated and modular Python pipeline. Beginning with CSV input, the system efficiently loads records, computes meaningful metrics (averages, percentages, rankings, status), identifies events requiring attention, and generates clear visualizations. This demonstrates that practical campus analytics can be achieved using standard Python libraries when the process is divided into clear, modular stages.

Moreover, the modular design (separate files for data modeling, analysis, charts, and main logic) significantly enhances maintainability and usability, allowing individuals with basic Python knowledge to understand and extend the system. The design emphasizes accessibility and practicality, ensuring that event performance and feedback monitoring is not limited to complex enterprise tools. Overall, the project meets its primary objective by delivering a functional, data-driven Campus Event Management & Feedback Analyzer that is both practical and educationally valuable.

REFERENCES

1. pandas Development Team. pandas: Powerful data structures for data analysis. Available at: https://pandas.pydata.org/

2. Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in Science & Engineering. Available at: https://matplotlib.org/

3. Python Software Foundation. Python Language Reference. Available at: https://www.python.org/

4. Project source code and files. Available at the project repository / OneDrive link provided by the student.